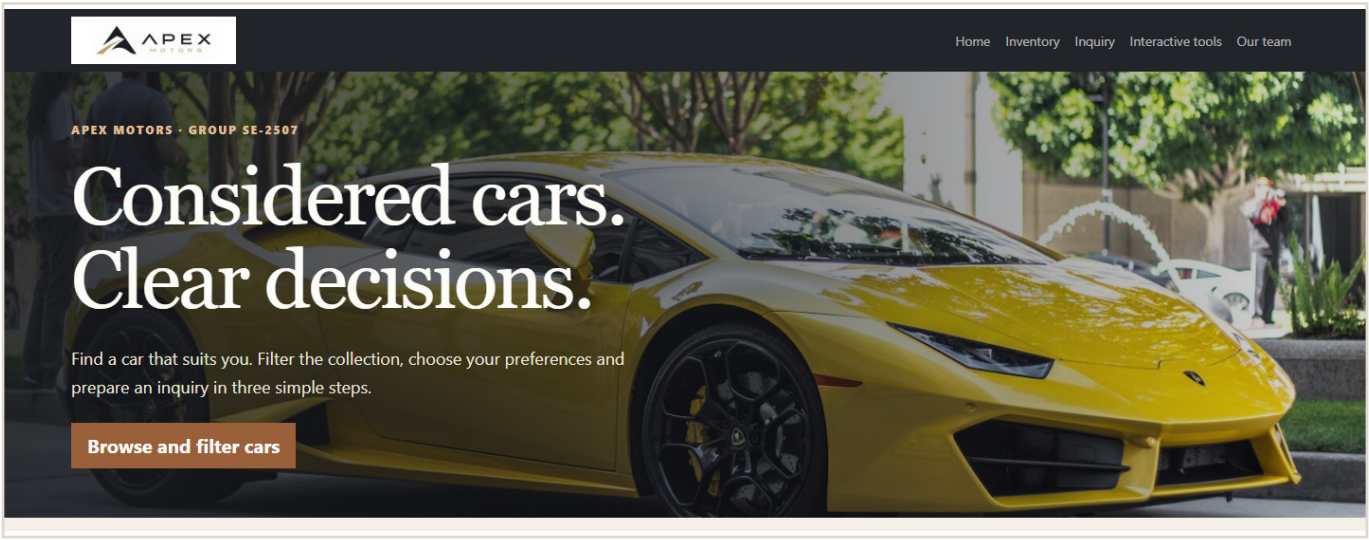
WEB TECHNOLOGIES · ASSIGNMENT 5

JavaScript and DOM Manipulation.

A single-page dealership with DOM updates, event handling, objects, arrays, callbacks, sound and animation.



The home page retains the cream/copper design and full-width vehicle hero.

Team: Apex Motors **Group:** SE-2507
Members: Arsen, Kuttibay, Farabi, Muhammad
Prepared: 29 October 2026
Build: HTML, custom CSS and three JavaScript files

Objective. Build a single-page dealership site with all required JavaScript interactions directly available on the main page. Keep the team's familiar colours, vehicle hero, cards and photographs.

Website: https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/
Repository: https://github.com/Vertex-blip/Car-dealership-website



Arsen



Kuttibay



Farabi



Muhammad

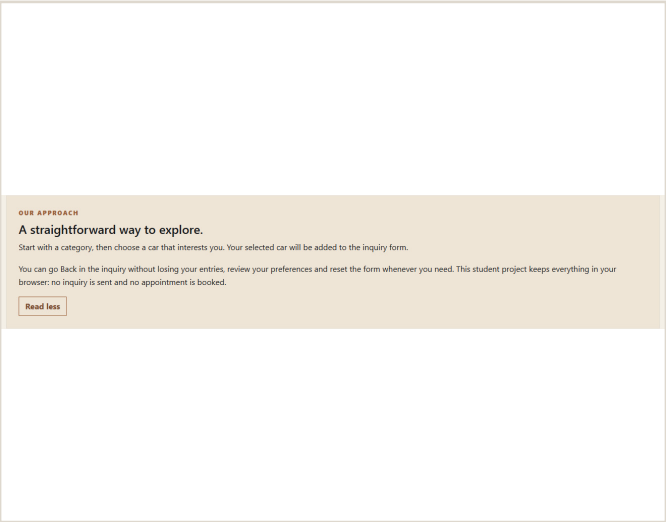
DOM updates and styling

Our approach contains Read More for the content-visibility option. Interactive tools provides a direct background-colour change for the dynamic-styling option.

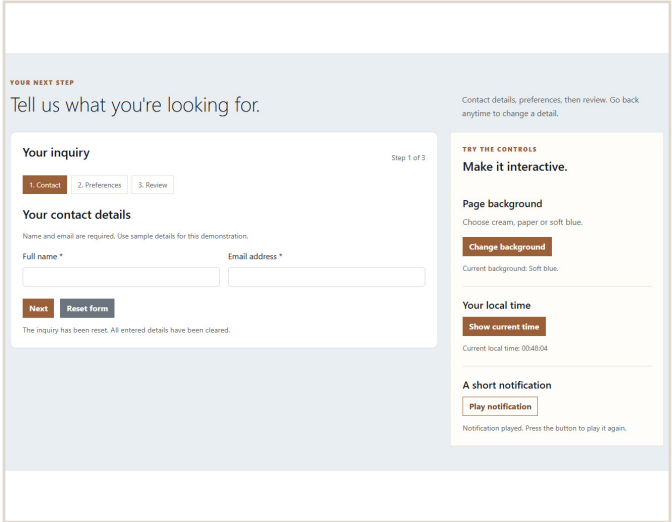
1. Select the Read More button and extra text; attach a click listener.

2. Change style.display and the button state to reveal or hide the extra content.

3. Cycle a small array of colours and assign the selected colour to body.style.backgroundColor.



Our approach: additional content revealed.



Interactive tools: background changed by JavaScript.

js/site.js

```
107      readMoreButton.setAttribute('aria-expanded', 'false');
108      readMoreButton.addEventListener('click', () => {
109        const expanded = readMoreButton.getAttribute('aria-expanded') !== 'true';
110        readMoreContent.style.display = expanded ? 'block' : 'none';
111        readMoreButton.setAttribute('aria-expanded', String(expanded));
112        readMoreButton.textContent = expanded ? 'Read less' : 'Read more';
113        if (expanded) window.Apex.animate(readMoreContent);
114      });
115    }
```

js/site.js

```
126      backgroundStatus.textContent = `Current background: ${colors[currentColor].name}.`;
127      backgroundButton.addEventListener('click', () => {
128        currentColor = (currentColor + 1) % colors.length;
129        const color = colors[currentColor];
130        document.body.style.backgroundColor = color.value;
131        backgroundStatus.textContent = `Current background: ${color.name}.`;
132      });
133    }
```

Actual source: js/site.js

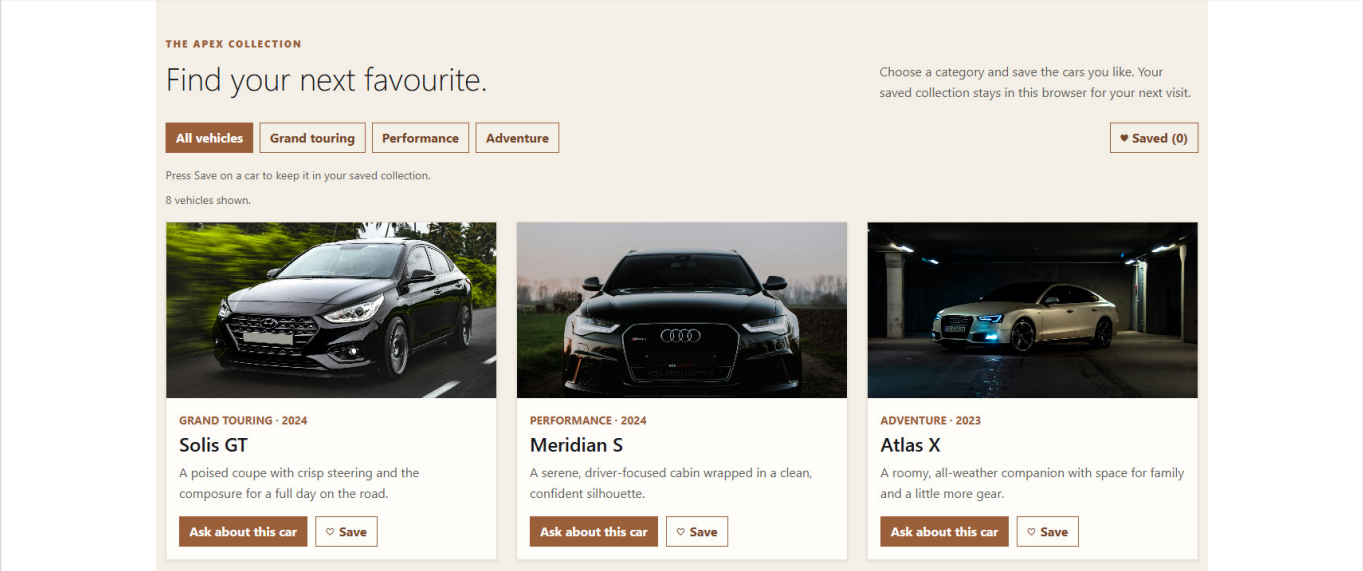
Objects, arrays and vehicle cards

The eight-car collection retains the original three listings and adds Camaro, Huracan, Panamera, Expedition and LaFerrari. Cards use structured objects and local photographs.

1. Store each vehicle's name, category, photo and description in an object with a label method.

2. Keep all eight vehicle objects in one array and create DOM cards in an explicit for...of loop.

3. Read object properties and insert content with textContent rather than user-written HTML.



Eight-car collection: first row shown; the result count covers all eight cars.

js/site.js

```
  6      return {
  7        id, name, category, categoryLabel, year, image, alt, description,
  8        label() {
  9          return `${this.categoryLabel} · ${this.year}`;
 10        }
 11      };
```

js/site.js

```
 15    const vehicles = [
 16      createVehicle('solis', 'Solis GT', 'touring', 'Grand touring', 2024,
```

js/site.js

```
 25      createVehicle('camaro', 'Chevrolet Camaro', 'performance', 'Performance', 2018,
 26        'images/car-04.jpg', 'Turquoise Chevrolet Camaro coupe parked in a desert landscape',
 27        'Bold coupe styling and an athletic stance for a drive that feels like an
    occasion.'),
```

js/inventory.js

```
113      const fragment = document.createDocumentFragment();
114      for (const vehicle of selectedVehicles) fragment.append(makeCard(vehicle));
```

Actual source: js/site.js, js/inventory.js

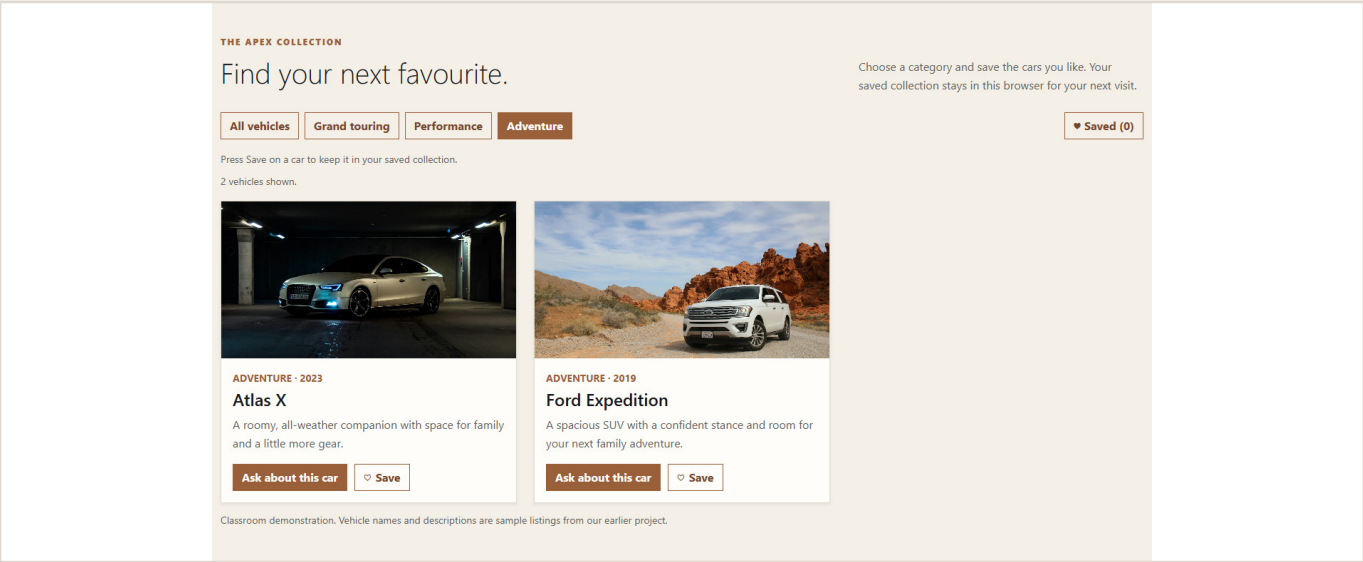
Category filtering

Category buttons filter the existing collection without a page reload. The result count and visible cards respond to the selected category.

1. Listen for category button clicks and read the selected category.

2. Use a switch statement to choose the matching category condition.

3. Pass that condition to filter, then render the resulting array and update the result count.



Inventory: a selected category updates the collection.

js/inventory.js

```
54    function matchesCategory(vehicle) {
55      // Each switch branch controls which objects reach the visible card grid.
56      switch (category) {
57        case 'touring': return vehicle.category === 'touring';
58        case 'performance': return vehicle.category === 'performance';
59        case 'adventure': return vehicle.category === 'adventure';
60        default: return true;
61      }
62    }
63
64    function updateSaveButton(button, vehicle) {
65      const saved = savedIds.includes(vehicle.id);
```

js/inventory.js

```
109    function render(animate = true) {
110      // filter is higher-order: it receives matchesCategory as its predicate callback.
111      const selectedVehicles = vehicles.filter(matchesCategory)
112        .filter(vehicle => !savedOnly || savedIds.includes(vehicle.id));
113      const fragment = document.createDocumentFragment();
114      for (const vehicle of selectedVehicles) fragment.append(makeCard(vehicle));
115      if (!selectedVehicles.length) {
116        const message = savedIds.length
```

Actual source: js/inventory.js

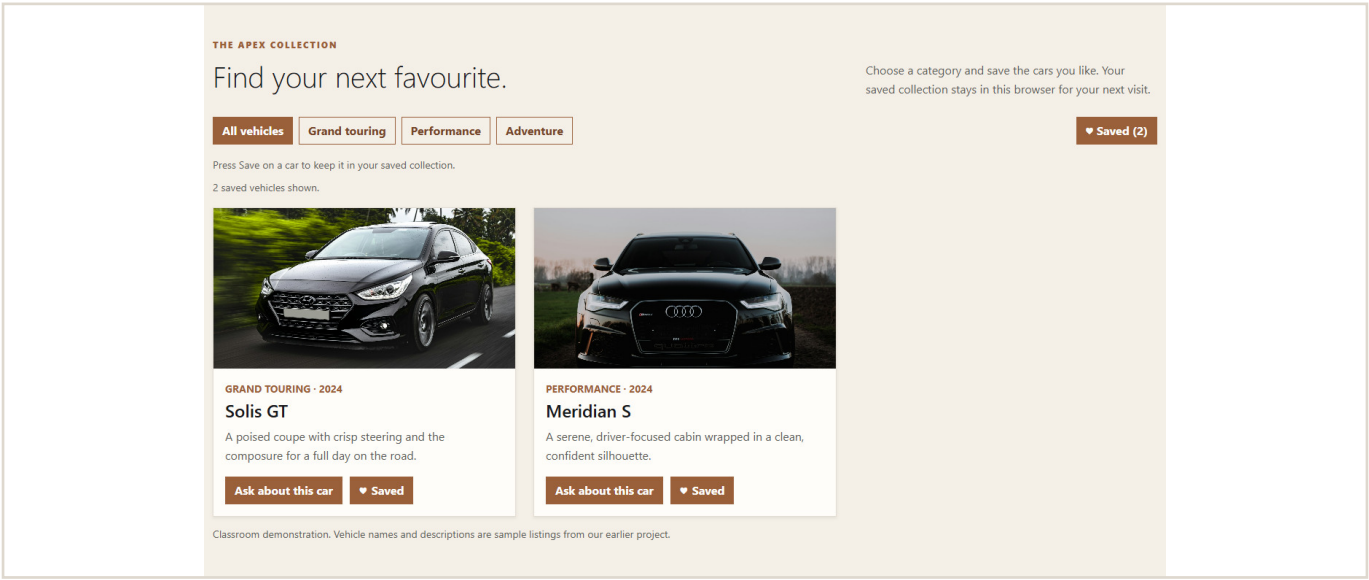
Favorites saved in this browser

Favorites extends the dealership theme with a personal shortlist. JavaScript updates each card's saved state and the Saved filter; localStorage retains vehicle IDs when available.

1. Use a card's Save button to add or remove that vehicle from the shortlist.

2. Serialize saved IDs to localStorage and restore valid IDs when the page opens.

3. Choose Saved to see the shortlist; show clear feedback for an empty list or unavailable storage.



Inventory: saved vehicles selected with the Saved filter.

js/inventory.js

```
 28        const ids = JSON.parse(stored || '[]');
 29        if (!Array.isArray(ids)) return [];
 30        // Keep only known vehicle IDs and discard duplicates or stale data.
 31        return [...new Set(ids.filter(id => vehicles.some(vehicle => vehicle.id === id)))];
```

js/inventory.js

```
 37    function storeSavedIds() {
 38      try {
 39        window.localStorage.setItem(storageKey, JSON.stringify(savedIds));
 40        storageAvailable = true;
 41      } catch {
 42        storageAvailable = false;
 43      }
 44      return storageAvailable;
 45    }
```

js/inventory.js

```
148      const wasSaved = savedIds.includes(vehicle.id);
149      savedIds = wasSaved ? savedIds.filter(id => id !== vehicle.id) : [...savedIds,
     vehicle.id];
150      const persisted = storeSavedIds();
151      savedStatus.textContent = `${vehicle.name} ${wasSaved ? 'removed from saved cars.' :
     'saved.'}`
152        + (persisted ? '' : ' This change could not be kept for your next visit; it lasts
     until this page is reloaded.');
```

Actual source: js/inventory.js

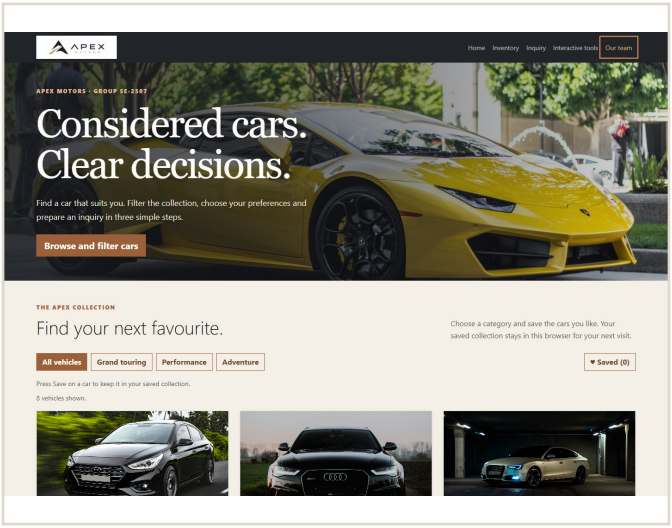
Button and keyboard events

The five navigation links lead to sections on this page and support the required arrow-key focus movement. Buttons also update the page through event listeners.

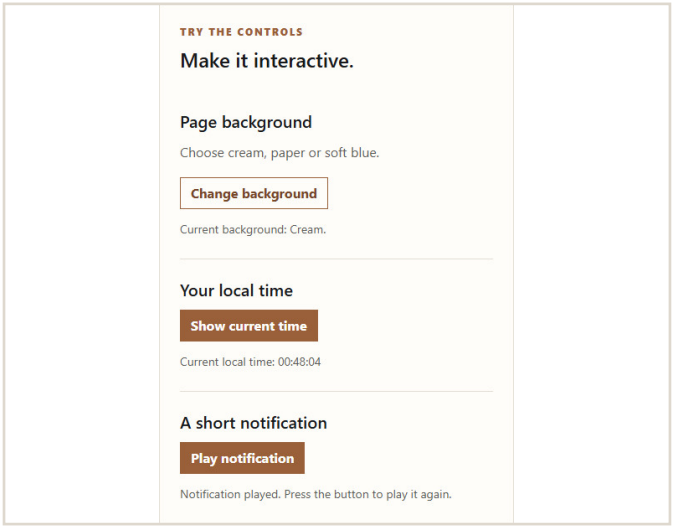
1. Select the navigation links and register a keydown listener on the menu.

2. Detect arrow keys, calculate the next or previous index with wraparound, and focus that link.

3. Use the time button's click callback to show a newly formatted current time.



A menu link receives keyboard focus.



Interactive tools: working time and sound actions.

```
js/site.js
 71      navigation.addEventListener('keydown', event => {
 72        const current = links.indexOf(document.activeElement);
 73        if (current === -1) return;
 74        let next;
 75        switch (event.key) {
 76          case 'ArrowRight':
 77          case 'ArrowDown': next = (current + 1) % links.length; break;
 78          case 'ArrowLeft':
 79          case 'ArrowUp': next = (current - 1 + links.length) % links.length; break;
 80          case 'Home': next = 0; break;
 81          case 'End': next = links.length - 1; break;
js/site.js
 92        event.preventDefault();
 93        links[next].focus();
 94      });
js/site.js
137    if (timeButton && timeOutput) {
138      timeButton.addEventListener('click', () => {
139        timeOutput.textContent = `Current local time: ${new Date().toLocaleTimeString('en-
     GB')}`;
140        window.Apex.animate(timeOutput);
141      });
```

Actual source: js/site.js

06 · CALLBACK INTERACTION

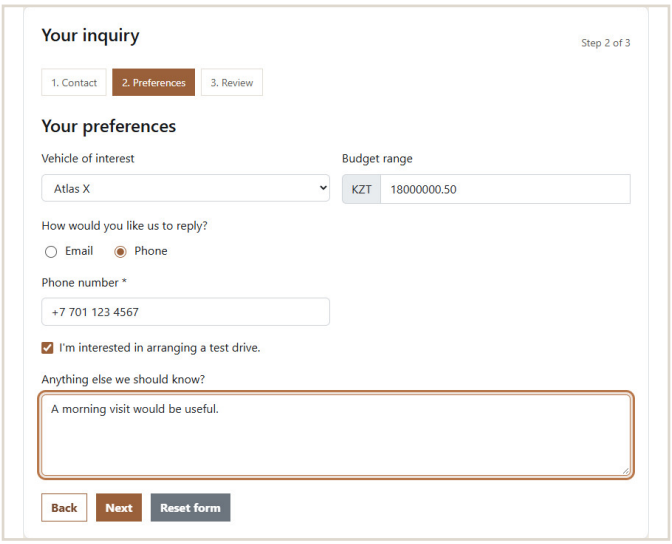
One inquiry, three steps

The multi-step form is the chosen callback option. The original inquiry fields are grouped into contact details, preferences, and review. Its eight vehicle choices come from the same collection array.

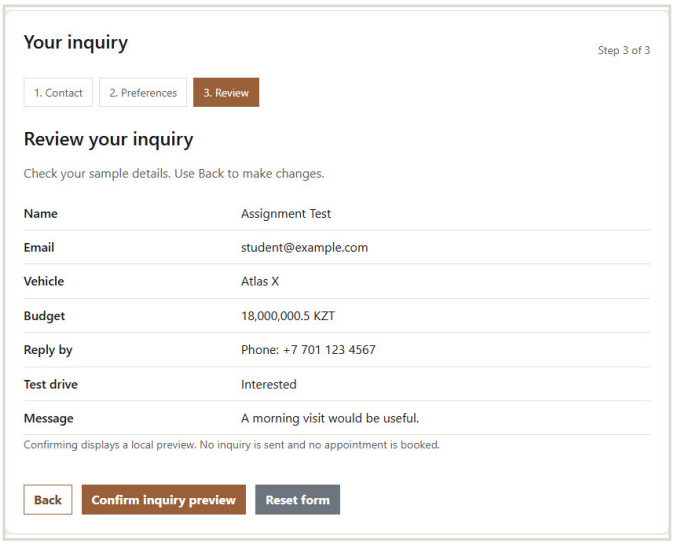
1. Keep the current step in state and validate it before moving forward.

2. Pass the step-display function as a callback to the navigation function.

3. Hide or reveal fieldsets without recreating their inputs, so Back retains entered values.



Step 2: inquiry preferences.



Inquiry: review before confirmation.

```
js/booking.js
144    function navigateStep(direction, onStepChange) {
145      if (direction > 0 && !validateStep(currentStep)) {
146        status.textContent = 'Please correct the highlighted fields before continuing.';
147        return;
148      }
149      const nextStep = Math.max(0, Math.min(2, currentStep + direction));
150      onStepChange(nextStep);
151    }
152    nextButton.addEventListener('click', () => navigateStep(1, showStep));
153    previousButton.addEventListener('click', () => navigateStep(-1, showStep));
154
js/booking.js
117    function showStep(index, moveFocus = true) {
118      currentStep = index;
119      steps.forEach((step, position) => { step.hidden = position !== currentStep; });
120      previousButton.hidden = currentStep === 0;
js/booking.js
 23    const vehicleOptions = document.createDocumentFragment();
 24    for (const vehicle of window.Apex.vehicles) {
 25      const option = document.createElement('option');
 26      option.value = vehicle.id;
 27      option.textContent = vehicle.name;
 28      vehicleOptions.append(option);
 29    }
 30    vehicleSelect.append(vehicleOptions);
```

Actual source: js/booking.js

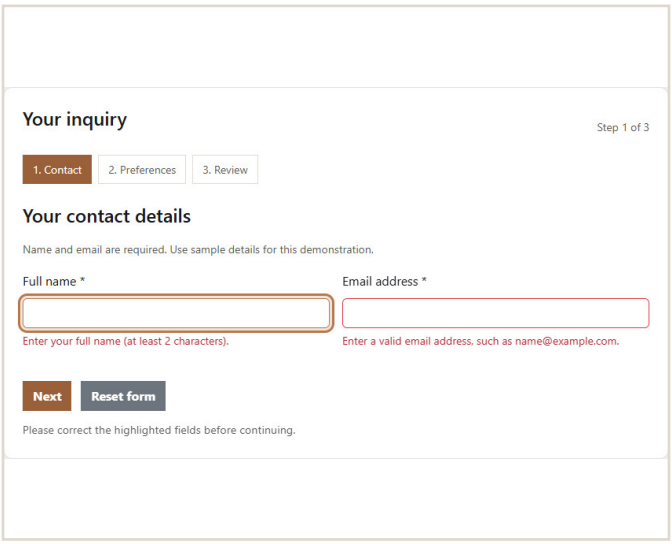
Validation, review and reset

The form checks required contact values and the user's preferences, then displays a local preview result. It does not send personal information to a server.

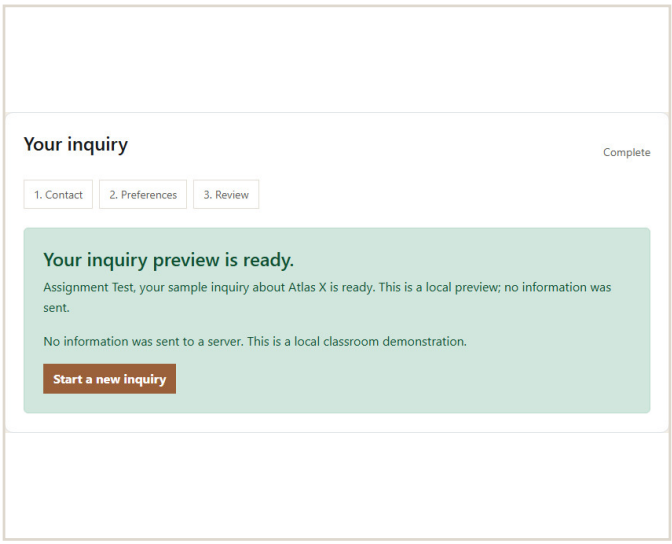
1. Prevent default submission, display field errors, and focus the first invalid control.

2. Build the review from the entered values and show confirmation without reloading.

3. Reset all fields, error messages, and step state when the user starts again.



Missing or invalid values receive clear feedback.



A completed inquiry preview remains on the page.

js/booking.js

```
 77    function validateStep(index, focusFirst = true) {
 78      let firstInvalid = null;
 79      for (const id of fieldsByStep[index]) {
 80        if (!setError(id, fieldError(id)) && !firstInvalid) firstInvalid =
    document.getElementById(id);
 81      }
 82      if (focusFirst && firstInvalid) firstInvalid.focus();
 83      return !firstInvalid;
 84    }
 85
 86    function clearResolvedStatus() {
 87      // Read validity without adding new error messages before a navigation attempt.
```

js/booking.js

```
155    function clearValues() {
156      form.reset();
157      form.querySelectorAll('input, textarea').forEach(input => {
158        if (input.type === 'radio') input.checked = input.id === 'replyEmail';
159        else if (input.type === 'checkbox') input.checked = false;
160        else input.value = '';
161      });
162      vehicleSelect.value = '';
163      fieldsByStep.flat().forEach(id => setError(id, ''));
164      document.getElementById('inquirySummary').replaceChildren();
```

Actual source: js/booking.js

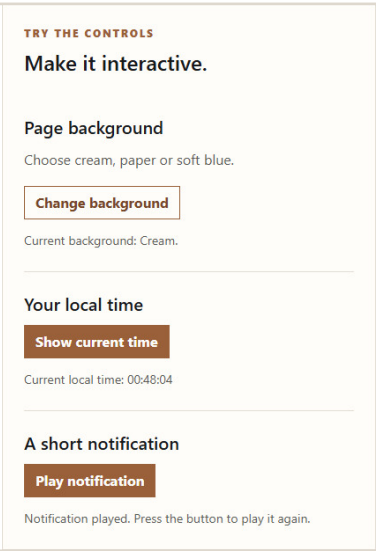
Sound and event-triggered animation

Sound and motion are attached to intentional user actions. The sound file is included locally and playback is reused instead of creating overlapping clips.

1. Load one Audio player and start it only after the notification button is clicked.

2. Pause and rewind the player before each playback; report playback failures clearly.

3. Animate content, inventory or form changes and respect the reduced-motion preference.



Interactive tools: user-triggered notification sound and feedback.

js/site.js

```
148      const sound = new Audio('assets/chime.wav');
149      sound.preload = 'auto';
150      sound.volume = 0.4;
151      let requestNumber = 0;
152      async function playChime() {
153        const currentRequest = ++requestNumber;
154        sound.pause();
155        sound.currentTime = 0;
156        try {
157          await sound.play();
158          if (currentRequest !== requestNumber) return;
159          soundStatus.textContent = 'Playing the notification sound.';
```

js/site.js

```
 45      animate(element) {
 46        if (!element || !element.animate
 47            || window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;
 48        animations.get(element)?.cancel();
 49        const animation = element.animate(
 50          [{opacity: 0, transform: 'translateY(10px)'}, {opacity: 1, transform:
    'translateY(0)'}],
 51          {duration: 260, easing: 'ease-out'}
 52        );
 53        animations.set(element, animation);
```

Actual source: js/site.js

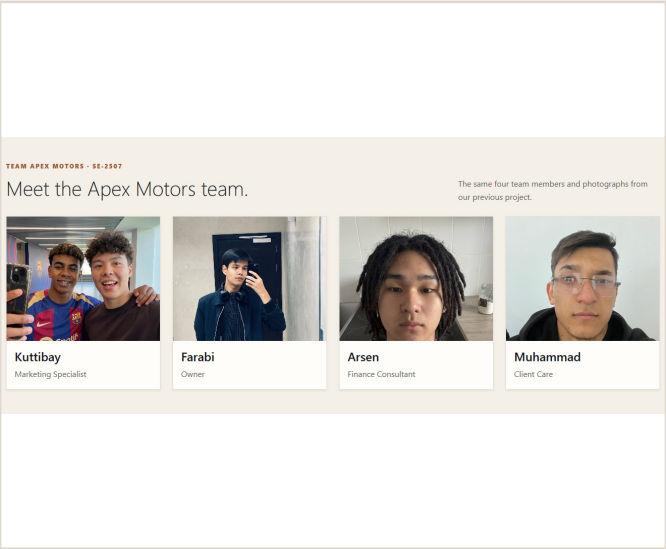
One page, separate source files

HTML, CSS and JavaScript are kept separate. All required features share one main page. The colour palette, hero, square cards and photographs continue the team's earlier design.

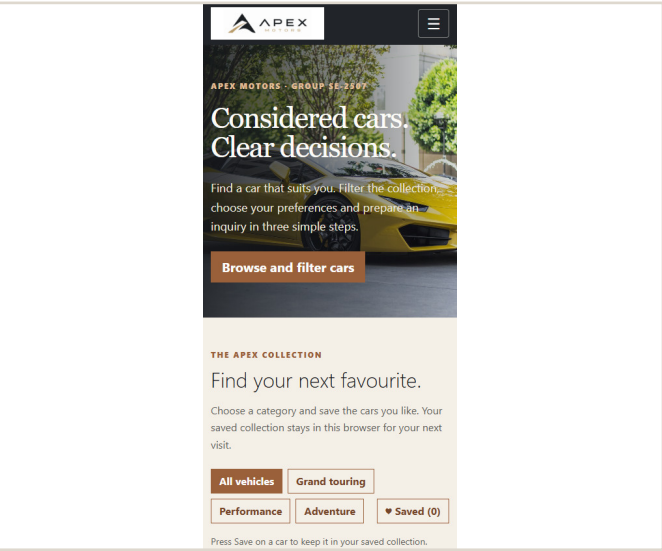
1. Link the custom stylesheet and three external deferred scripts from index.html.

2. Use section anchors, visible labels, alternative text, keyboard focus and a custom mobile menu.

3. Check every interaction and the four original team portraits at desktop and phone widths.



The same four team members and portraits.



Responsive main page at phone width.

index.html

```
12      <link rel="stylesheet" href="styles.css?v=collection-a5" />
13      <script src="js/site.js?v=collection-a5" defer></script>
14      <script src="js/inventory.js?v=collection-a5" defer></script>
15      <script src="js/booking.js?v=collection-a5" defer></script>
16    </head>
17    <body>
18      <a class="skip-link" href="#main-content">Skip to main content</a>
19      <header class="site-header">
```

styles.css

```
 2  :root {
 3    --apex-ink: #17191c;
 4    --apex-copper: #9a603a;
 5    --apex-cream: #f5f0e7;
 6    --apex-paper: #fffdf9;
 7    --apex-muted: #5c5c59;
 8    --apex-border: #e3ddd3;
 9  }
10
```

Actual source: index.html, styles.css

Testing and reflection

We checked the following interactions in the browser:

• **Collection and styling.** All eight cars appear; categories update the cards and result count. Read More reveals extra text, and the background button changes the page colour.

• **Favorites.** Save and remove update each card and the saved count. The Saved filter shows the shortlist, which is restored after a reload when browser storage is available.

• **Inquiry.** The vehicle choices match the collection. Next validates inputs, Back retains values, and review, confirmation and reset work through the three steps.

• **Keyboard and layout.** Arrow keys move focus between menu links. The mobile menu opens and closes, and the sections fit desktop and phone widths.

• **Sound and motion.** The notification button plays the local sound. Repeated clicks restart one player without overlapping clips, and content changes trigger animation.

Deployed website:
https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/

Challenges and solutions. The cards and inquiry list must show the same models, so both are generated from one vehicle array. Form validation checks the current step and focuses its first invalid field before allowing the user to continue. Storage access uses try...catch so Favorites remains usable on the open page if the browser cannot save the shortlist.

Reflection. A shared collection means adding a vehicle requires one data entry to update both cards and form choices. Passing showStep as a callback separates step navigation from display; hiding the existing fieldsets retains the entered values. Saving vehicle IDs as JSON keeps the shortlist small, and checking restored IDs against the collection prevents stale entries. Contact details remain outside this storage.

Submission and defense. Each member submits the PDF and ZIP individually and explains the implementation during the practice lesson.